🗄️ CampusPulse — Backend, Database & Server Actions Technical Manual

Version: 1.0.0 (Production)

Classification: Backend Engineering & Database Architecture

Database: MongoDB Atlas (Mongoose 9.9.4)

Architecture: Next.js 16 Server Actions + NextAuth.js v4

Last Updated: September 2026

---

1. Database Architecture & Connection Pooling

CampusPulse connects to **MongoDB Atlas** using **Mongoose ODM**. Serverless environments (like Netlify or Node microservices) frequently spawn and destroy execution containers. To prevent database connection starvation and connection exhaustion, CampusPulse implements a **Global Connection Caching Singleton** in `src/lib/db/connection.ts`.

Connection Caching Pattern:

```
// src/lib/db/connection.ts
import mongoose from "mongoose";
import dns from "dns";

// Fix for Node.js SRV record resolution behind restricted local DNS / corporate WiFi
try {
  dns.setServers(["8.8.8.8", "8.8.4.4", "1.1.1.1"]);
} catch {
  // Ignored in sandboxed runtimes
}

interface MongooseCache {
  conn: typeof mongoose | null;
  promise: Promise<typeof mongoose> | null;
}

declare global {
  var mongooseCache: MongooseCache | undefined;
}

const cached: MongooseCache = global.mongooseCache ?? { conn: null, promise: null };
if (!global.mongooseCache) global.mongooseCache = cached;

async function dbConnect(): Promise<typeof mongoose> {
  const uri = process.env.MONGODB_URI;
  if (!uri) throw new Error("Please define MONGODB_URI in your environment.");

  if (cached.conn) return cached.conn;

  if (!cached.promise) {
    cached.promise = mongoose.connect(uri, {
      bufferCommands: false,
      maxPoolSize: 10,
    });
  }

  try {
    cached.conn = await cached.promise;
  } catch (e) {
    cached.promise = null;
    throw e;
  }

  return cached.conn;
}

export default dbConnect;
```

2. Exhaustive Data Schema Index (Mongoose Models)

CampusPulse defines 7 distinct collections in the `campuspulse` database:

1. `User.ts` (`users` collection)

Stores authentication credentials, university departments, and permission roles.

- `_id`: ObjectId (Primary Key)
- `name`: `String` (Full name)
- `email`: `String` (Unique, lowercase normalized, indexed)
- `passwordHash`: `String` (Bcrypt encrypted hash with 10 salt rounds)
- `department`: `String` (e.g., Computer Science, Information Technology, AI & Data Science)
- `role`: `String` (Enum: `"student"` | `"admin"`, default: `"student"`)
- `createdAt`: `Date` (Timestamp)

2. `Event.ts` (`events` collection)

Central event document containing schedule, capacity locks, attendee tokens, and media vaults.

- `title`: `String` (Headline title)
- `slug`: `String` (Unique URL slug, e.g., `hackpulse-2026`)
- `description`: `String` (Markdown-ready description)
- `category`: `String` (Enum: `"Technical"`, `"Cultural"`, `"Sports"`, `"Workshop"`)
- `tags`: `[String]` (Search keywords, e.g., `["AI", "Hackathon", "Coding"]`)
- `eventDate`: `Date` (Scheduled date & time)
- `venue`: `String` (Campus hall, room number, or grounds)
- `maxCapacity`: `Number` (Seat limit for automated capacity blocking)
- `status`: `String` (Enum: `"draft"`, `"upcoming"`, `"ongoing"`, `"completed"`, `"cancelled"`)
- `organizerId`: `ObjectId` (Ref to `User`)
- `registeredUsers`: `Array`
- `userId`: `ObjectId` (Ref to `User`)
- `qrCodeToken`: `String` (Unique UUID for scannable entry pass verification)
- `attended`: `Boolean` (True once checked in at the door)
- `registeredAt`: `Date`
- `mediaGallery`:
- `photos`: `[String]` (Public URLs or Data URIs)
- `rawClips`: `[String]` (Video URLs or embed links)
- `reportData`:
- `chiefGuest`: `String` (Name of dignitary)
- `objectives`: `String` (Educational/institutional goals)
- `outcomes`: `String` (Impact report summary)

- `facultyCoordinator` : `String` (Supervising professor)
- `createdAt` : `Date`

3. `FeedbackForm.ts` (`feedbackforms` collection)

Dynamic survey builder schema supporting multiple question formats.

- `eventId` : `ObjectId` (Ref to `Event`, indexed)
- `title` : `String` (Survey title)
- `description` : `String`
- `questions` : `Array`
- `questionId` : `String`
- `questionText` : `String`
- `type` : `String` (Enum: `"rating"` (1–5 stars), `"text"`, `"choice"`)
- `options` : `[String]` (For multiple choice questions)
- `required` : `Boolean`
- `isActive` : `Boolean` (Enables/disables student submission window)
- `createdAt` : `Date`

4. `FeedbackSubmission.ts` (`feedbacksubmissions` collection)

Stores individual student responses and sentiment scores.

- `formId` : `ObjectId` (Ref to `FeedbackForm`)
- `eventId` : `ObjectId` (Ref to `Event`)
- `userId` : `ObjectId` (Ref to `User`, optional for anonymous submissions)
- `responses` : `Array`
- `questionId` : `String`
- `answer` : `Schema.Types.Mixed` (String or Number)
- `submittedAt` : `Date`

5. `EventQnA.ts` (`eventqnas` collection)

Live interactive student Q&A forum with upvote tracking.

- `eventId` : `ObjectId` (Ref to `Event`, indexed)
- `userId` : `ObjectId` (Ref to `User`)
- `userName` : `String` (Attendee name)
- `question` : `String` (Student question text)
- `upvotes` : `[ObjectId]` (Array of user IDs who upvoted to prevent duplicate votes)
- `upvoteCount` : `Number` (Calculated for fast sorting)
- `answer` : `String` (Official reply from coordinator/speaker)
- `isPinned` : `Boolean` (Stuck to top of live board)
- `answeredAt` : `Date`

- `createdAt`: `Date`

6. `SiteSettings.ts` & 7. `SocialBroadcast.ts`

Platform-wide institutional announcements, hero headlines, footer contact info, and social media deep links.

---

3. Next.js Server Actions Engine (`src/actions/`)

CampusPulse utilizes Next.js Server Actions for all mutations, eliminating fragile client API routes:

Action File	Key Functions	Operations Executed
`events.ts`	`rsvpEventAction()`	Verifies capacity, generates unique QR token, appends user to `registeredUsers`
`events.ts`	`cancelRsvpAction()`	Pulls user from registration list and recalculates available capacity
`events.ts`	`verifyQrCodeAction()`	Decodes token, returns ticket holder details and real-time validity status
`events.ts`	`toggleAttendanceAction()`	Marks student `attended: true/false` upon QR scan at the door
`feedback.ts`	`submitFeedbackAction()`	Validates required questions, stores responses, updates aggregate ratings
`qna.ts`	`postQuestionAction()`	Publishes question to live event forum
`qna.ts`	`upvoteQuestionAction()`	Atomic `$addToSet` / `$pull` on `upvotes` array to prevent double-voting
`qna.ts`	`pinQuestionAction()`	Admin toggle to pin critical announcement questions
`analytics.ts`	`getAdminAnalyticsAction()`	Computes total students, overall attendance %, category breakdown
`upload.ts`	`uploadMediaAction()`	Writes to `public/uploads` or returns Base64 Data URI on serverless hosts

---

4. Authentication & Role-Based Access Control (RBAC)

Security is enforced at the function level using helper utilities in `src/lib/auth/rbac.ts`:

```
// Example: RBAC enforcement inside a Server Action
import { requireAdmin, requireAuth } from "@/lib/auth/rbac";

export async function createEventAction(formData: FormData) {
  // 1. Will throw Unauthorized error if not signed in as 'admin'
  const user = await requireAdmin();

  // 2. Perform privileged mutation safely...
}
```

- **Student Role:** Can view events, RSVP, view personal tickets, submit feedback, post and upvote questions.
- **Admin Role:** Full CRUD over events, access to `/admin` dashboard, feedback analytics, media manager, executive report printer, and attendance toggling.